Assessment 1	Frontend design and usability (React)
Due date	11.59 pm (AEST) Sunday 26 July, 2026 (Week 2)
Weighting	20%
Word count/length	1,200 words equivalent (±10%; the reference list is not included in the word count but in-text citations are)
SILOs	• Design and develop web applications using JavaScript (SILO 1). • Use modern software engineering tools to build and deploy robust code for scalable web sites (SILO 4).
Graduate capabilities	• INQUIRY AND ANALYSIS - Creativity and Innovation • INQUIRY AND ANALYSIS - Critical Thinking and Problem Solving • INQUIRY AND ANALYSIS - Research and Evidence-Based Inquiry
Generative AI use	**Full AI:** AI use is accepted for specific tasks such as drafting text, and refining and evaluating work. You are expected to critically evaluate and modify any AI-generated content included. The AI Assessment Scale resource provides guidance on the appropriate use of GenAI tools in assessments and explains what is expected in each category.
Feedback	Written feedback will be provided within two weeks of submission. Feedback will support improvement for Assessment 2 (backend implementation and RSS capability).
Individual	This is an individual assessment.
Short extension	**Eligible:** This assessment is eligible for a short extension. To apply, visit Assessment 1: 3-day short extension and add yourself to the extension group.

Purpose

This assessment is the first stage of a larger project that will continue across the subsequent assessments. Assessment 1 focuses specifically on front-end design and usability, with the aim of establishing a clear, responsive, and user-centred interface for the RSS Server and LMS application. At this stage, you are not required to send RSS content or implement backend feed processing. Instead, the emphasis is on creating the frontend experience for the server application, including how users will navigate, view, and interact with the system. In industry, frontend developers are expected to build interfaces that are not only functional, but also intuitive, accessible, and aligned with the needs of end users. This task prepares you to apply React, usability principles, and responsive design practices in the context of a real web application.

MUST BE CREATED FROM: npx create-next-app .

Task details

Design and implement a usability-focused frontend for an RSS Server feeding into an LMS, supported by a written justification of your design and architectural decisions. For Assessment 1, the work should concentrate on the user interface only. You may develop the interface using the blog-based content structure introduced in Module 4 Part 2 as a temporary stand-in for RSS data, allowing you to focus on layout, navigation, visual design, and usability before backend integration is introduced in Assessment 2.

Web pages

The frontend should include the following pages:

- **Home** — the main landing page, with a brief introduction to the project and links to the other pages.
- **About** — explains what the project is, confirms that Assessment 1 is frontend only, and briefly describes the future RSS Server and LMS connection.
- **Feeds / Posts** — displays blog-style sample content from Module 4 Part 2 in a card or list layout with titles, dates, summaries, and read-more links.
- **Settings** — provides interface controls such as light/dark mode and optional layout preferences.

Instructions

You are required to:

1. **Design and implement a React frontend** that includes:
    - component-based architecture
    - state management
    - responsive UI design
    - a navigation bar or tab bar
    - a header and footer (footer - has your name and student number, header has the assessment title)
    - an About page with your name, student number, and a short video explaining how to use the website
    - a hamburger menu or kebab menu for compact navigation, with options such as About and Settings.
2. **Apply usability and accessibility principles**, including:
    - navigation clarity
    - accessible colour contrast and readable typography
    - keyboard navigation and ARIA support where appropriate
    - user interaction design that supports quick scanning of feed content
    - light and dark themes where appropriate
    - theme preference saved using a cookie or local storage
    - hide/show behaviour for responsive menu and content sections.
3. **Represent the RSS-to-LMS workflow** through screens or interactive views that show how content will be sourced, displayed, and organised for users.
4. **Include interactive frontend features** that demonstrate progress from the weekly labs, such as:
    - breadcrumbs and structured navigation
    - dynamic pages or interactive links
    - local storage for user preferences such as theme or menu state
    - clear visual feedback for menu and navigation actions.
5. **Provide a written justification (approx. 1,200 words)** covering:
    - design decisions
    - component structure and scalability
    - UX considerations and improvements
    - trade-offs made in the frontend design
    - how the interface supports the RSS Server and LMS use case.
6. **Technical expectations:**
    - Use React best practices.
    - Apply modular and reusable components.
    - Ensure code readability and maintainability.
    - Demonstrate a professional frontend suitable for later backend integration.
    - Use Next.js v22+ as the project framework.

Project continuity

This assessment forms the foundation for the broader project and will continue into the later assessments. Assessment 1 is intentionally focused on front-end design, usability, and user experience. Assessment 2 will introduce the server component and add RSS feed capability, enabling the application to accept and process live RSS content. This staged approach allows the frontend to be developed first using the blog-based material from Module 4 Part 2 as a temporary stand-in, before the full backend RSS functionality is implemented in the next stage.

Resources and readings relevant to the assessment

The following schedule shows the progress expected across the early modules and labs:

- **Module 1:** Project setup (Next.js v22+), with the Next.js App Router. Do not submit standalone HTML/JavaScript files.
- **Module 2:** Cookies and cloud setup

- **Module 3:** Carousel, hamburger menu, themes, local storage, React, and hide/show blocks
- **Module 4:** Advanced React, list rendering, and dynamic pages
- **Module 5 and beyond:** API routes, Docker, database, testing, observability, cloud-native components, and authentication for later assessments

Other relevant resources include:

- React documentation
- WCAG accessibility guidelines
- Lecture and tutorial materials on frontend design, responsive layouts, component architecture, and usability

Assessment criteria

The assessment will be marked using the criteria below. Students should ensure their submission demonstrates both functional implementation and clear explanation of design decisions. The rubric focuses on the frontend experience only for Assessment 1; RSS feed sending and backend server functionality are introduced in Assessment 2.

Criterion	Description for students	Max score
User Interface	Include clear frontend components such as a navigation bar, header, footer, and About page. The interface should be easy to understand, visually organised, and suitable for an RSS Server frontend.	4
Themes	Include light mode and dark mode. The theme should be easy to switch, visually consistent, and applied across the application. Other theme-related improvements are also encouraged.	3
Hamburger Menu	Include a hamburger menu or kebab menu for responsive navigation. The menu should use a CSS transform or animation to show interaction and should work well on smaller screens.	3
Interactive Frontend Views + Functionality	Provide interactive frontend content that demonstrates navigation and layout for the server interface. This may include hide/show blocks, dynamic pages, breadcrumbs, reusable content areas, and local storage for settings such as theme or menu state.	5
Code Quality and GitHub	Submit a well-organised GitHub repository with several commits, separate branches for major features, a clean main branch, no `node_modules` folder, and an up-to-date README file.	5
Video Demonstration	Record a 3 to 8 minute walkthrough showing your student ID, face, and voice, then demonstrate the application, navigation, theme switching, and key frontend functionality.	Not separately scored

Submission details

Submit:

- A zip file containing your project code and GitHub repository link. Remove `node_modules` before uploading.
- A video recording (3 to 8 minutes maximum) showing your student ID, face, and voice, and demonstrating your application, code, navigation, themes, and key frontend functionality.

In keeping with La Trobe University policy, all assignments are to be submitted in Moodle via Turnitin.

To be accepted, your assessment submission **must** generate a similarity score (you are responsible for checking this). If your submission does not generate a similarity score, it cannot be checked for plagiarism and therefore **will not be marked**.

AI acknowledgement

If AI use is acceptable in this assessment, you must complete and submit an 'AI acknowledgement' with your work. Failure to submit an acknowledgement when required may constitute a breach of academic integrity. You will find the AI acknowledgement on the Assessments page.

References

- Minimum five academic or industry sources
- APA 7th ed. style

Last modified: Friday, 10 July 2026, 12:17 PM

← Discussion: Assessments Q&A

Jump to activity

Assessment 1: Marking criteria and rubric →